



Shift Left Without the Pain

Turning Security into a DevOps Habit with Open Source

Abhinav Bharadwaj R

FOSSUnited Bengaluru · April Meetup 2026



Open Source

We deploy fast.

But are we deploying vulnerabilities?

80%

of breaches involve
a known vulnerability

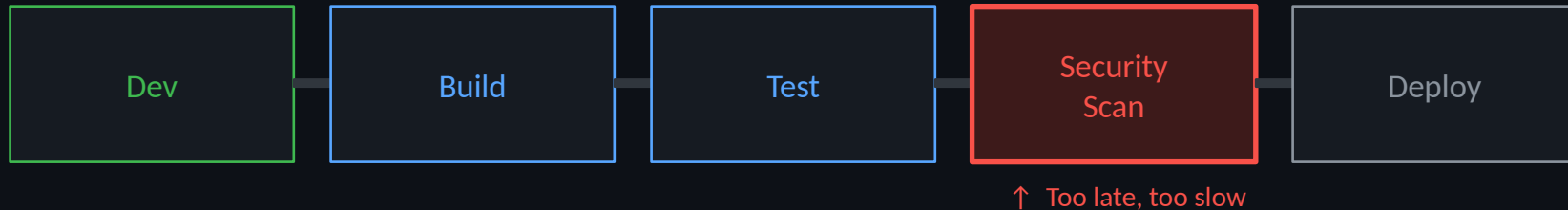
60 days

average time to fix
a critical CVE

70%

of issues found
in production are fixable
earlier

Security is Late, Manual, or Ignored



Vulnerabilities found post-deploy

Fixing in prod is 10× costlier than dev



Manual security reviews are a bottleneck

One security team, hundreds of PRs



Scanning is an afterthought

Scheduled scans, not pipeline-native



Developers aren't trained to think security

Security is 'someone else's problem'

Why Shift Left Fails

(And why teams give up)



Too many tools

SAST, DAST, SCA, container scanning, IaC scanning... each needing a different config and expertise



Alert fatigue

Thousands of findings, most low-risk. Developers tune out. False positives kill trust.



No developer feedback loop

Security findings show up in a separate dashboard nobody reads during a sprint



Complex setup

Signing up for enterprise scanners, API keys, dashboards — barriers before first scan



What if security could be
simple and already in your pipeline?

No new dashboards. No new accounts. No YAML sprawl.

Just one open-source binary.

Trivy

Open Source · by Aqua Security · Apache 2.0

```
brew install trivy | apt install  
trivy
```



Container Images

- › OS packages (Alpine, Debian, Ubuntu)
- › Language libs (npm, pip, go, gem)
- › Dockerfile misconfigs



Source Code / Repos

- › Dependency files (package.json, Pipfile)
- › Secret detection (API keys, tokens)
- › SBOM generation (CycloneDX, SPDX)

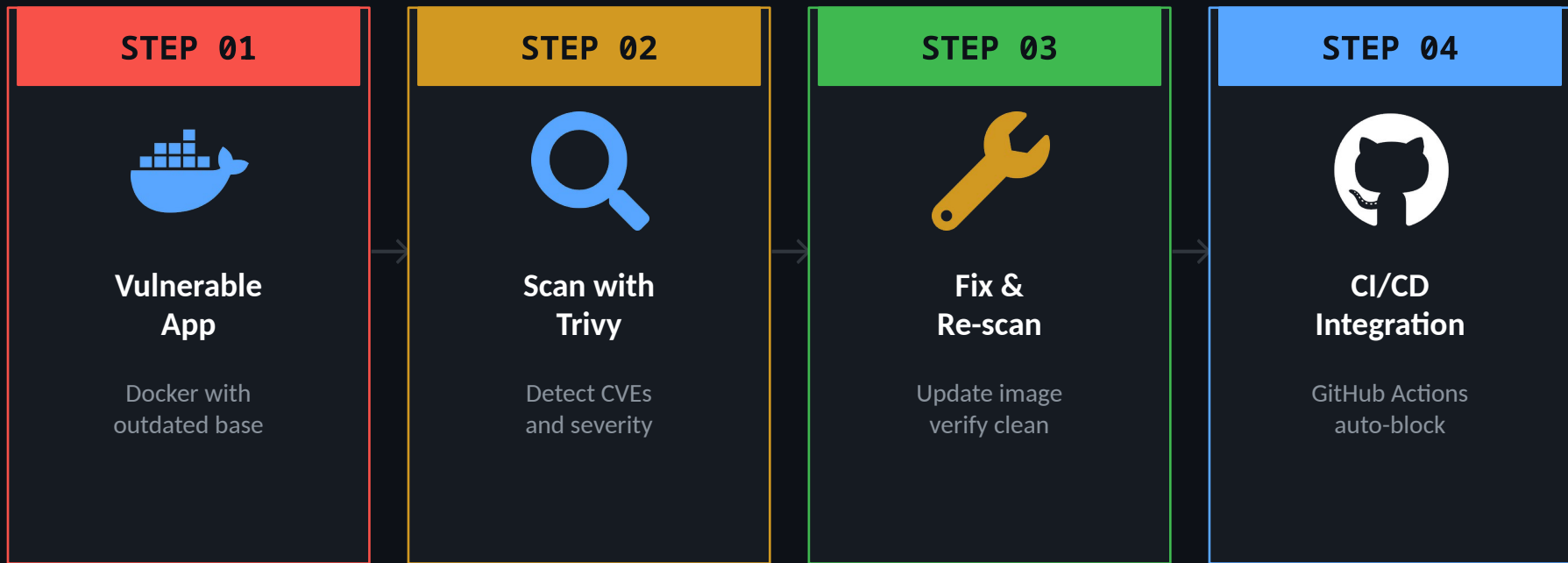


IaC & Config

- › Terraform, Kubernetes manifests
- › Helm charts
- › Docker Compose, CloudFormation

Demo: Scan → Fail → Fix → Automate

4 steps • ~10 minutes



A simple Flask app on an old base image

Dockerfile

```
FROM python:3.6-alpine

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
CMD ["python", "app.py"]
```

requirements.txt

```
flask==0.12.2
requests==2.18.0
pyyaml==3.12
```



Why it's dangerous:

python:3.6-alpine → EOL since Dec 2021, 50+ known CVEs

flask 0.12.2 → Multiple HIGH severity CVEs

```
$ docker build -t demo-app:vulnerable .
```

One command to see everything

```
$ trivy image demo-app:vulnerable
```

Library	Vulnerability	Severity	Installed	Fixed
python	CVE-2022-37454	CRITICAL	3.6.15	3.11.0
flask	CVE-2018-1000656	HIGH	0.12.2	0.12.3
requests	CVE-2023-32681	HIGH	2.18.0	2.31.0
pyyaml	CVE-2020-14343	CRITICAL	3.12	5.4
urllib3	CVE-2023-45803	MEDIUM	1.21.1	2.0.7

2 CRITICAL

2 HIGH

1 MEDIUM

Two changes. Zero vulnerabilities.

BEFORE

```
FROM python:3.6-alpine

WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

requirements.txt

```
flask==0.12.2
requests==2.18.0
pyyaml==3.12
```

AFTER

```
FROM python:3.12.13-alpine

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

```
flask==3.0.3
requests==2.31.0
pyyaml==6.0.1
```

✓ trivy image demo-app:fixed → 0 vulnerabilities found

Fail the pipeline on HIGH or CRITICAL

```
name: Trivy Security Scan
on:
  push:
    branches: [ "main" ]
jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t app:latest .

      - name: Run Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'app:latest'
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'
          ignore-unfixed: true
```



exit-code: '1'

Pipeline fails if HIGH or CRITICAL CVEs found



ignore-unfixed: true

Skip CVEs with no patch available — reduces noise



PR-level blocking

Vulnerable code never merges to main

Reality Check

What Trivy won't solve for you



False positives exist

Especially in distroless images or Go binaries. Use `.trivyignore` to suppress known-safe findings with justification.



Not all CVEs are exploitable

A CVE in a networking lib doesn't matter if your app doesn't touch the network. Context matters.



Trivy doesn't catch logic bugs

It scans known CVEs and misconfigs — not SQL injection or auth bypass in your own code. You still need code review.



Governance still needs a human

Who decides which CVEs are acceptable risk? That policy has to come from your team, not a tool.

Best Practices to Avoid the Pain

1



Start small

Scan one service first. Don't boil the ocean.

2



Fail selectively

Block only HIGH+CRITICAL. Log the rest.

3



Use ignore with justification

Every .trivyignore entry should have a comment.

4



Pin base image digests

FROM python:3.12@sha256:... for reproducible builds.

5



Scan on PR, not just main

Catch issues before they land in the codebase.

6



Keep Trivy updated

CVE databases lag. Run the latest scanner weekly.

Key Takeaways



Security scanning doesn't need to be complex — one binary, one CI step.



Trivy gives you actionable results: what's vulnerable, and exactly what version fixes it.



Fail selectively. Block CRITICAL/HIGH, log the rest. Don't let noise kill adoption.



Open source is production-ready. Trivy is used by Google, GitHub, and AWS internally.



Shift left works when friction is low. The simpler the setup, the more likely teams will use it.

Thank You!



github.com/abhinavbharadwajr/trivy-foss-demo

Abhinav Bharadwaj R · FOSSUnited Bengaluru · April 2026

